

Test Design Report: CodePop

Version: 1

Team: SocialDrinkers (3)

- [Test Design Report: CodePop](#)
 - [1. Introduction: Testing Journey & Mindset](#)
 - [2. Testing Approach \(Draft 1: "How"\)](#)
 - [2.1 Backend Isolation Testing](#)
 - [2.2 Frontend API & Interaction Testing](#)
 - [3. System-Level \(End-to-End\) Testing](#)
 - [3.1 Automated Multi-Instance Orchestration](#)
 - [3.2 Manual User Acceptance Tests \(UAT\)](#)
 - [4. Challenges & Areas of Concern](#)
 - [4.1 Distributed System Testing](#)
 - [4.2 Distributed User Authentication](#)
 - [4.3 Current Worries](#)
 - [5. Code Coverage & Metrics](#)
 - [5.1 Backend Coverage](#)
 - [5.2 Frontend Coverage](#)
 - [6. Final Testing Execution Plan \(Draft 2: "How"\)](#)
 - [6.1 How We Will Execute Existing Tests](#)
 - [6.2 High-Risk Areas and Expected Failure Modes](#)
 - [6.3 How We Will Respond to Issues](#)
 - [6.4 Coverage Targets and Final Demo E2E Rehearsal](#)
 - [Final Presentation: System-Level \(End-to-End\) Rehearsal Steps](#)
 - [7. Appendices \(Screenshots & Logs\)](#)
 - [7.1 Automated Multi-Instance Orchestration Log](#)
-

1. Introduction: Testing Journey & Mindset

Our approach to testing CodePop has been guided by four core principles:

- **Comprehensive Coverage:** Ensuring that all critical endpoints are tested thoroughly.
- **Resilience:** Testing how the system handles unexpected conditions and failures gracefully.
- **Distributed Consistency:** Ensuring project stability even when users are roaming from their Home Server.
- **Secureness:** Protecting against unauthorized access through rigorous cryptographic verification.

Because CodePop is a decentralized Peer-to-Peer (P2P) network, our testing journey evolved significantly during development. We initially started with standard Django unit tests to verify individual components. However, as the project evolved into a multi-node architecture system, our mindset shifted. We realized that "success" wasn't just passing local tests, but ensuring the network itself could survive peer discovery and data synchronization across multiple nodes without crashing or producing inconsistent state.

2. Testing Approach (Draft 1: "How")

To ensure stability across our decentralized architecture, we adopted a tiered testing strategy that addresses the application at different levels of granularity:

- **Isolation Layer (Backend Unit):** Using Django `unittest` and `APITestCase` to validate core models (Inventory, Orders, Users) and security logic in a controlled environment.
- **Interaction Layer (Frontend API):** Utilizing a custom TypeScript test runner (`api_tests.ts`) to verify that the frontend models correctly interface with the backend API and handle real-world data shapes.
- **Orchestration Layer (System Integration):** Employing a custom multi-instance framework to simulate complex P2P conditions, including cross-server authentication and regional data aggregation.

This layered approach ensures we catch issues early in isolation while still verifying the "big picture" network behavior that our users rely on.

2.1 Backend Isolation Testing

- **Framework:** Django `unittest` and `REST Framework APITestCase`.
- **Strategy:** Using `unittest.mock` to simulate network traffic for fast logic verification without requiring a live peer network.
- **Scope:**
 - **Model & Logic:** Validating core data models (Inventory, Orders, Users) and business logic.
 - **P2P Security:** Verifying RS256 asymmetric signature validation and public key exchange logic in isolation.
 - **Authentication:** Testing JWT claim verification and proxy authentication logic.
- **Automation:** Integrated into the development workflow (`python manage.py test`) to catch regressions early.

2.2 Frontend API & Interaction Testing

- **Framework:** Custom TypeScript Test Runner (`api_tests.ts`).
- **Strategy:** End-to-end API verification from the frontend's perspective.
- **Scope:**
 - Authentication flows (Login, Register, Profile Management).
 - Business logic (Drink creation, Ordering, Inventory management).
 - Cross-cutting features (Chatbot, Notifications, Revenue).
 - Admin/P2P controls (Server discovery, Master list access).
- **Automation:** Can be executed via `npm run ts-node models/tests/api_tests.ts` to verify frontend-to-backend compatibility.

3. System-Level (End-to-End) Testing

While the unit and API testing can account for how the system works in isolation, it cannot accurately simulate the complexities of the distributed nature of the system. This phase orchestrates the deployment of multiple live server instances, real database interactions, and end-to-end user interactions, ensuring that we can accurately simulate real-world scenarios of how production environments will behave. Unlike the isolation testing mentioned in 2.1, this layer uses no mocks for network traffic, relying instead on real HTTP

requests between live instances. To ensure CodePop functions before final deployment, we implement a high-fidelity, system-wide testing strategy.

3.1 Automated Multi-Instance Orchestration

Our core integration verification is handled by a sophisticated Python orchestration script:

`full_p2p_automated_test.py`. This script automates a complex, multi-node environment by performing the following:

- **Dynamic Infrastructure:** Spins up **three independent Django servers** (A, B, and C) on dynamic ports and creates **three separate PostgreSQL databases** to ensure zero state-leakage between nodes.
- **Auto-Discovery & Sync:** Verifies that servers can cross-register using RSA public keys and that the global **MasterList** (user registry) correctly propagates across all three nodes.
- **Cross-Server Flows:** Tests "Roaming" user scenarios where a user registered on Server A logs in and performs actions (profile updates, preference creation) on Server B, which are then proxied back to their home server.
- **Full-Stack Features:** Beyond authentication, the script verifies:
 - **Stripe Integration:** Simulates complete order flows, including mock Stripe payment intents and webhook handling.
 - **Machine Proxying:** Tests the proxying of status data from a simulated "Pseudo Machine" server to the central dashboard.
 - **Global Aggregation:** Verifies that a Super Admin can fetch unified revenue and inventory metrics aggregated from all active regional servers.
 - **Fault Tolerance:** Proactively terminates a server node to verify that the rest of the network handles the offline state gracefully.

This script ensures that the decentralized architecture of CodePop remains robust and that every inter-server communication path is verified before deployment. (see [appendix 7.1](#))

- **Steps to Reproduce:**
 1. `source codepop_virtual_enviroment/bin/activate`
 2. `python codepop_backend/integration_tests/full_p2p_automated_test.py`
- **Expected Outcome:** All 16 verification steps (Auth, Proxying, Stripe, Aggregation) return "SUCCESS", which can be verified by the final summary log in the terminal output.

3.2 Manual User Acceptance Tests (UAT)

- **Scenario 1: User Registration & Login**
 - **Action:** Create a user, log in, and verify JWT token receipt.
 - **Outcome:** Redirect to dashboard, user session persisted.
- **Scenario 2: Ordering a Drink**
 - **Action:** Select a soda, add add-ins, and complete "mock" checkout.
 - **Outcome:** Inventory decremented, order visible in history.
- **Scenario 3: Cross-Server Soda Discovery**
 - **Action:** Browse sodas while connected to a visiting server.
 - **Outcome:** Sodas from the home server are listed correctly via proxy.

4. Challenges & Areas of Concern

4.1 Distributed System Testing

The most significant testing challenge was verifying roaming user functionality across different server nodes. CodePop’s requirement for location-agnostic access, where a user can connect to any regional server and maintain full functionality, necessitated a complex proxying architecture rather than simple database replication. Testing this was particularly difficult because it required simulating real-time network requests between instances to ensure that local and proxied data (like user preferences and order history) were merged correctly without introducing state-mismatches.

4.2 Distributed User Authentication

Testing the distributed authentication system without manual intervention was a significant hurdle and served as the primary driver for our Automated Multi-Instance Orchestration suite. Automating these cross-node handshakes was the only way to ensure consistent behavior without the risk of human error.

Specifically, we encountered several technical challenges with asymmetric key (RS256) verification:

- **PEM Formatting Sensitivity:** We found that subtle differences in newline handling (e.g., `\n` vs `\r\n`) or trailing whitespace during database storage would cause signature verification to fail silently. To mitigate this, we had to implement a robust "key cleaning" pipeline that strips all whitespace before deriving fingerprints for the Server-Token handshake, ensuring consistent verification across different environments.
- **Bootstrapping Synchronization:** Managing the initial "discovery" phase—where a new server must securely register its public key with a peer before it can issue valid JWTs—required precise timing to prevent race conditions that would otherwise break the trust chain.

By creating the Multi-Instance Orchestration suite, we could verify that this integral part of the whole system worked properly, even in a production-like environment. This prevented cases where localized testing would succeed, even if the system wouldn't work in real-world scenarios.

4.3 Current Worries

There are several areas of concern that require more testing and refinement:

- **Edge Cases in Network Latency during P2P Discovery:** Ensuring that the system can handle a high level of latency without delaying the rest of the system startup.
- **Race Conditions in Inventory:** Ensuring the system works in cases of multiple requests for inventory changes at once.
- **Edge Cases in AI Logic:** Ensuring the AI system doesn't fulfill a request for an order that has already been fulfilled.

5. Code Coverage & Metrics

Layer	Unit Tests	Integration Tests	Overall Coverage (Combined)
Backend	~70%	~50% (Overall) / 90%+ (P2P)	~70%
Frontend	0%	~20% (API/Model Layer)	~20%

5.1 Backend Coverage

We currently have around 70% overall code coverage for the backend endpoints (33 out of 47 unique paths) and core services. Our testing strategy heavily prioritized the **P2P networking and security logic**, which maintains 90%+ coverage via the `full_p2p_automated_test.py` orchestration suite. This suite simulates a multi-node network to verify RSA key exchange, token proxying, and cross-server synchronization. Standard Django unit tests in `tests.py` handle model validation and CRUD operations for the local database.

5.2 Frontend Coverage

Frontend coverage is currently estimated at **~20%**, focused entirely on the **API and Model layer**. We utilize a custom TypeScript test runner (`api_tests.ts`) that exercises the frontend's communication with the backend, covering authentication, order placement, and inventory retrieval. While the data-handling logic is verified, the React UI components and pages (presentation layer) currently have 0% automated unit test coverage. Manual testing serves as the primary verification for the visual and interactive elements of the application.

6. Final Testing Execution Plan (Draft 2: "How")

This section explains how we will run final testing before presentation. Some tests were built earlier; we will now use them as our official release checks.

6.1 How We Will Execute Existing Tests

- **Backend Unit/API Baseline (Django + DRF):**
 - We will run `tests.py` and `tests_p2p.py` as baseline regression tests.
 - Current baseline: **65 tests** (56 backend + 9 P2P/auth).
- **Frontend API/Model Baseline (TypeScript runner):**
 - We will run `models/tests/api_tests.ts` as the main frontend API test suite.
 - Current baseline: **73 checks** across auth, drinks/orders, inventory, chatbot, and admin/P2P routes.
- **Distributed Integration Baseline (Multi-instance orchestration):**
 - We will run `full_p2p_automated_test.py` as the final system-level gate.
 - Pass condition: all **16 orchestration steps** succeed.

6.2 High-Risk Areas and Expected Failure Modes

- **Distributed trust boundary (highest risk):**
 - Roaming login, refresh proxying, and logout blacklisting are most likely to fail first.
- **API response-shape drift:**
 - Some endpoints may return different object/array shapes.
- **Permission edge cases:**
 - We will verify role boundaries (including expected **403** responses).
- **Inventory update semantics:**
 - PATCH/update behavior may vary by endpoint and needs close checking.
- **Operational timing issues:**
 - Peer discovery and handshake timing may expose latency/race issues.

6.3 How We Will Respond to Issues

- **Authentication or P2P failures:**
 - Isolate whether the issue is token claims, key verification, or proxy routing.
 - Add a targeted regression test before closing the bug.
- **Contract/response mismatches:**
 - Align API and frontend expectations, then update assertions.
- **Permission regressions:**
 - Treat as release-blocking and rerun role-based tests.
- **Inventory/order consistency defects:**
 - Reproduce with fixed payloads, patch, and rerun backend + frontend suites.

6.4 Coverage Targets and Final Demo E2E Rehearsal

- **Coverage targets for final sprint validation:**
 - Backend target: about **~70%** endpoint/core coverage.
 - Frontend target: about **~20%** automated API/model coverage (UI unit coverage remains low).

Final Presentation: System-Level (End-to-End) Rehearsal Steps

1. Run the automated orchestration suite

- Action:
 - Execute `python codepop_backend/integration_tests/full_p2p_automated_test.py` from the project root.
- Expected outcome:
 - All checks pass and the final summary reports no failed steps.

2. Validate roaming authentication manually

- Action:
 - Authenticate on a non-home server with a user whose account belongs to a different server.
- Expected outcome:
 - Proxy login succeeds and role-based access is preserved.

3. Validate cross-server business flows

- Action:
 - Perform profile, drink, and order operations during a visiting-server session.
- Expected outcome:
 - Operations succeed and user data remains consistent across expected routes.

4. Validate aggregation and machine proxying

- Action:
 - Execute super-admin aggregation routes and pseudo-machine status flows.
- Expected outcome:
 - Aggregated metrics return correctly; degraded/offline scenarios do not crash the system.

5. Capture final-presentation evidence

- Action:
 - Capture screenshots/logs during the full rehearsal run.
- Expected outcome:
 - Evidence includes orchestration success, roaming login success, cross-server order proof, and super-admin aggregation proof.

Screenshots are included in Appendix 7 where available; missing items will be captured during final rehearsal.

7. Appendices (Screenshots & Logs)

7.1 Automated Multi-Instance Orchestration Log

```
===== Initializing Port Discovery =====
Using dynamic ports: Server A = 8050, Server B = 8051, Machine = 8052, Server C = 8053

===== Starting FULL AUTOMATED P2P INTEGRATION TEST (AUTO-DISCOVERY) =====
Generating unique RSA keypair for Server A...
Generating unique RSA keypair for Server B...
Generating unique RSA keypair for Server C...
Generating unique RSA keypair for Server Malicious...

[1/16] Setting up database and self-identification...
Ensuring PostgreSQL database 'p2p_test_a' exists...
Ensuring PostgreSQL database 'p2p_test_b' exists...
Ensuring PostgreSQL database 'p2p_test_c' exists...
Database 'p2p_test_b' not found. Creating...
Database 'p2p_test_c' not found. Creating...
Database 'p2p_test_a' not found. Creating...
Database 'p2p_test_b' created.
Database 'p2p_test_c' created.
Database 'p2p_test_a' created.

[2/16] Launching background servers...
Spawning Server A on port 8050...
Spawning Server B on port 8051...
Spawning Server C on port 8053...
Spawning Pseudo Machine Server on port 8052 (Test Mode: True)...
Waiting for servers to be ready (timeout 20s)...
Server A is READY.
Server B is READY.
Server C is READY.

[3/16] Performing Cross-Registration via AUTO-DISCOVERY...

[4/16] Creating users across all servers and syncing...
Triggering MasterList sync from Server A...
Verifying MasterList sync on Server C...
SUCCESS: MasterList correctly synchronized (proven by successful roaming logins on Server C).

[5/16] TESTING AUTHENTICATION FLOWS...
Test A: Correct Password Proxy Login (B -> A)...
SUCCESS: Proxy logins worked and returned authoritative tokens from Server A via cookies.
Test A.1: Using Proxied Token on Visiting Server (B)...
SUCCESS: Server B correctly verified and accepted Server A's proxied token.
Test A.2: Role-Based Access Control (RBAC) on Visiting Server (B)...
SUCCESS: Server B correctly granted 'super_admin' access and denied 'customer' access based on P2P tokens.
Test B: Incorrect Password Proxy Login (B -> A)...
SUCCESS: Server correctly rejected wrong password.

[6/16] TESTING CROSS-SERVER JWT VERIFICATION...
Fetching valid token from Server A...
Test C: Using Server A's Token on Server B (Cross-Verification)...
SUCCESS: Server B verified Server A's token using Server A's Public Key.
Verified Identity: traveler_joe
Test D: Using Maliciously Signed Token on Server B...
SUCCESS: Server B correctly blocked malicious token (403).
Test D.1: Using an Expired Token on Server B...
SUCCESS: Server B correctly rejected the expired token (403).

[7/16] TESTING TOKEN REFRESH AND LOGOUT PROXYING
Test E: Token Refresh on Home Server (A)...
SUCCESS: Home server correctly refreshed token via HttpOnly cookie.
Test F: Token Refresh Proxy (B -> A)...
SUCCESS: Server B proxied the refresh request to Server A using cookies.
SUCCESS: Proxied access token (via cookie refresh) is valid on Server B.
Test G: Logout Proxy (B -> A)...
SUCCESS: Server B proxied the logout/blacklist request to Server A and cleared local cookie.
SUCCESS: Logout proxying and cookie clearing verified.

[8/16] TESTING CROSS-SERVER REGISTRATION
Test H: Registering a brand new user directly on Server B...
SUCCESS: Server B registered the new user and issued a token claiming ownership via cookies.

[9/16] TESTING ORDER CREATION AND STRIPE WEBHOOKS
Test I: End-to-End Order Creation and Payment Webhook...
SUCCESS: Payment success and Revenue creation verified.
Simulating Refund Webhook...
SUCCESS: Refund webhook and state transition verified.
SUCCESS: Full order creation, payment intent, and webhook flow passed.

[10/16] TESTING MACHINE STATUS PROXY
Test J: Fetch Machine Status from Proxy Endpoint...
SUCCESS: Machine proxy successfully fetched data from pseudo server.
Test J.1: Test Machine Offline Handling...
SUCCESS: Machine proxy gracefully handled offline machine (503).
Test J.2: Verify Machine 'Run Test' (F12) Flow...
Spawning Pseudo Machine Server on port 8052 (Test Mode: True)...
SUCCESS: Machine 'run-test' successfully cleared faults.

[11/16] Testing Data Proxying and Syncing (Roaming User)...
SUCCESS: Roaming user authenticated on Visiting Server B.
Testing Profile Patch Proxying (B -> A)...
SUCCESS: Profile update proxied correctly to Home Server.
Testing Preference Creation Proxying (B -> A)...
SUCCESS: Preference creation proxied correctly to Home Server.
Testing Order Creation Syncing (B -> A) with automatic Drink sync...
SUCCESS: Order and related Drinks successfully synced back to Home Server.
Testing Order History Merge (B reads local + remote A)...
SUCCESS: Order history merge successful. Found 3 orders.

[12/16] Populating extra Order and Revenue data for aggregate testing (via API)...
SUCCESS: Extensive Order & Revenue data populated via API across all servers.

[13/16] Testing Aggregate Views...
SUCCESS: Global Revenue Aggregate fetched successfully. Servers included: 3
SUCCESS: Data completeness verified (includes Server A, B, C)

[14/16] Testing Cross-Server Request Proxying...
SUCCESS: Cross-Region read (Aggregate inventory) worked for Logistics Manager.
SUCCESS: Home Server routing via proxy worked for Store Manager.

[15/16] Comprehensive RBAC Enforcement Tests...
SUCCESS: RBAC: Store Manager Isolation enforced (403 on aggregate).
SUCCESS: RBAC: Customer Restrictions enforced (403 on inventory report).
SUCCESS: RBAC: Admin Privilege Boundaries enforced (403 on global aggregate).
SUCCESS: RBAC: Repair Staff Access verified (200 on machines, 403 on revenue).

[16/16] Additional Resilience Tests (Fault Tolerance)...
Temporarily terminating Server C...
SUCCESS: System gracefully handled offline node with status 200.

===== Cleaning up processes and databases =====
Terminated background server process.
Terminated background server process.
Terminated background server process.
Dropping PostgreSQL database 'p2p_test_a'...
Database 'p2p_test_a' dropped.
Dropping PostgreSQL database 'p2p_test_b'...
Database 'p2p_test_b' dropped.
Dropping PostgreSQL database 'p2p_test_c'...
Database 'p2p_test_c' dropped.
Done.

-----
Ran 22 test scenarios in 44.198s

SUCCESS: VERIFICATION COMPLETE: ALL P2P Security and Auth Scenarios Passed.
```